

Real-Time Payment Fraud Detection for an E-Commerce Platform

Business Analytics Group Project — From Raw Data to a Deployable Model

Team: [Insert team member names]

Course: Business Analytics

Source: <https://github.com/Trungdn4-458311/FraudDetectionBE>

Abstract

Payment fraud causes substantial financial loss and erodes customer trust on e-commerce platforms. Acting as the risk-analytics function of a platform, this project answers a concrete business question: *which transactions are likely fraudulent, and how should the platform balance blocking fraud against approving legitimate orders without unnecessary friction?* We combine **two data sources**: the simulated mobile-money dataset **PaySim** (~6.36M transactions, fraud rate 0.129%) and **9 contextual features** generated in Python with Faker (account age, new-device flag, shipping/billing mismatch, IP distance, etc.). The pipeline covers exploratory data analysis (EDA), documented before/after cleaning, feature engineering, and training of **three classifiers** (Logistic Regression, Random Forest, XGBoost) over **two feature spaces** (Base/Full), evaluated with metrics suited to imbalanced data (PR-AUC, Recall, F1) plus a **cost-based metric** for selecting an operating threshold. A key methodological point: the 9 synthetic features are generated *conditioned on the label*, introducing **label leakage**; we quantify its effect through the Base-vs-Full comparison. Our best models (XGBoost/Random Forest) reach PR-AUC > 0.99, but we show that the near-perfect scores largely reflect the *easiness* of the data (PaySim is nearly separable via balance features) rather than genuine real-world generalization — an essential conclusion for an honest report.

Keywords: fraud detection; class imbalance; PaySim; synthetic data; label leakage; PR-AUC; cost-based threshold; XGBoost; Random Forest.

1 Introduction

An e-commerce platform processes thousands of transactions per minute, and even a fraud rate well under 5% translates into large absolute losses. We frame the task as **binary classification** on a **severely imbalanced** dataset: flag high-risk transactions in real time while limiting the number of legitimate customers blocked.

1.1 Business KPIs

We track three indicators:

- **Fraud rate** — the proportion of fraudulent transactions (measures imbalance and risk scale).
- **False-positive rate (FPR)** — the proportion of legitimate transactions wrongly blocked (measures customer friction).
- **Financial loss avoided** — the fraud amount stopped by the model (measures business value).

Across the full dataset the fraud rate is 0.129% (8,213 out of 6,362,620 transactions), with a **total money at risk** of $\sim 1.21 \times 10^{10}$ units and a mean of ~ 1.47 M per fraudulent transaction. This is the baseline against which Section 6.4 is compared.

1.2 Contributions and structure

The report follows the first five modules of the project: Section 2 (Module 1) describes the data and synthetic generation; Section 3 (Module 2) explores the data; Section 4 (Module 3) cleans it;

Section 5 (Module 4) engineers features; Section 6 (Module 5) trains, evaluates, and selects an operating threshold.

2 Data and Synthetic Generation (Module 1)

2.1 Source 1 — PaySim

PaySim is a simulated mobile-money dataset of 6,362,620 transactions with fields: **step** (time index, 1 step = 1 hour), **type** (PAYMENT, TRANSFER, CASH_OUT, CASH_IN, DEBIT), **amount**, origin/destination accounts (**nameOrig/nameDest**), pre/post balances of both parties, and the label **isFraud**.

2.2 Source 2 — 9 synthetic features (Faker)

The project requires additional contextual data generated in Python. We create 9 fields with the Faker library plus business logic, *injecting a fraud signal* to reflect realistic correlations (e.g., fraudulent accounts tend to be younger and use unfamiliar devices). A representative snippet is shown in Listing 1.

Listing 1: Generating transaction-level features conditioned on the label (excerpt).

```
1 fraud = df["isFraud"].to_numpy(bool)
2 # new device: higher probability if fraud
3 p_dev = np.where(fraud, 0.55, 0.08)
4 df["is_new_device"] = (rng.random(n) < p_dev).astype("int8")
5 # failed payment attempts (Poisson)
6 lam = np.where(fraud, 2.2, 0.12)
7 df["failed_payment_attempts"] = rng.poisson(lam).astype("int16")
```

All 9 fields are documented in a *data dictionary* (Table 1): column name, type, unit, valid range, and generation logic.

Table 1: Data dictionary of the 9 synthetic features (source 2).

Field	Type / Unit	Generation logic (summary)
account_age_days	int32 / days	Gamma; fraud accounts younger (mean ~44 vs ~500)
home_country	category	Uniform draw from a Faker country pool
device_fingerprint	SHA1	Sample from a pool of 5,000 hashes
hour_of_day	int16 / hour	step % 24
shipping_billing_mismatch	0/1	Bernoulli $p=0.40$ (fraud) / 0.03
is_new_device	0/1	Bernoulli $p=0.55$ (fraud) / 0.08
failed_payment_attempts	int16 / count	Poisson $\lambda=2.2$ (fraud) / 0.12
ip_billing_mismatch	0/1	Bernoulli $p=0.50$ (fraud) / 0.02
ip_billing_distance_km	float / km	Uniform(500,8000) if mismatch, else (0,80)

Label-leakage warning. Because the fields above are drawn *conditioned on isFraud*, their predictive power is partly *by design*. This is why, throughout the project, we compare two feature spaces (Base vs Full, Section 5) and interpret results cautiously (Section 6.5).

3 Exploratory Data Analysis (Module 2)

3.1 Class imbalance

The data is severely imbalanced: only 0.129% of transactions are fraudulent. Hence **Accuracy is inappropriate**; we prioritize PR-AUC, Recall, and F1.

3.2 Fraud by transaction type

Fraud occurs **only** in TRANSFER and CASH_OUT (Table 2). This is a key finding: we can pre-filter by transaction type to raise the positive density.

Table 2: Fraud distribution by transaction type.

Type	# Fraud	Fraud rate	# Transactions
CASH_OUT	4,116	0.184%	2,237,500
TRANSFER	4,097	0.769%	532,909
CASH_IN	0	0%	1,399,284
PAYMENT	0	0%	2,151,495
DEBIT	0	0%	41,432

3.3 Balance features

Many fraudulent transactions drain the origin account. We build two *balance-error* features to capture this anomaly:

$$\text{errorBalanceOrig} = \text{newbalanceOrig} + \text{amount} - \text{oldbalanceOrig}, \quad (1)$$

$$\text{errorBalanceDest} = \text{oldbalanceDest} + \text{amount} - \text{newbalanceDest}. \quad (2)$$

These separate fraud from legitimate transactions very well and later dominate the model’s feature importance (Section 6.3).

3.4 Fraud patterns in synthetic signals

The fraud rate spikes when risk flags equal 1 (new device, address mismatch, IP mismatch), and `account_age_days` / `ip_billing_distance_km` separate clearly by label — exactly as designed in Section 2 (hence the leakage caveat). In contrast, `hour_of_day` is derived from `step` and reflects a *genuine* temporal pattern of PaySim, with no leakage.

4 Data Cleaning (Module 3)

We clean the data under a *before/after documentation* principle, recording the number of fraud rows removed at every step (Table 3).

Table 3: Data-cleaning log.

Step	Rows removed	Fraud removed
Duplicate records (by original columns)	0	0
Invalid values (<code>amount</code> ≤ 0 / negative balance)	16	16
<code>amount</code> outliers (KEPT, not clipped)	0	0
Total (6,362,620 → 6,362,604)	16	16

Documented decisions.

- **No missing values or duplicates** — PaySim is simulated, high-quality data.
- **Outliers in amount are kept.** The high-value group (5.31% of the data under the 1.5×IQR rule) has a *fraud density* of 1.14% — about 8.8× the global rate (0.129%). Clipping the tail would delete rare positives; here an “outlier” is *signal*, not noise.
- **Merchant quirk.** Merchant destination accounts (`nameDest` starting with M, 33.8% of rows) always have zero destination balance by PaySim design, and no fraud targets a merchant. This is a data characteristic, not an error.

Note: all 16 removed rows were fraud (16/8,213 ≈ 0.19%) — a reminder that “label-blind” cleaning can still touch the rare class; the impact is negligible.

5 Feature Engineering (Module 4)

5.1 Type filtering and two feature spaces

Based on EDA, we keep only TRANSFER and CASH_OUT, yielding 2,770,393 samples with a positive density raised to 0.296% (from 0.129%). To measure the contribution of the synthetic data precisely, we build **two feature spaces**:

- **Base** — original PaySim transaction features plus “real” signals derived from the transaction sequence (balance errors, velocity). *No label leakage*.
- **Full** — Base + 7 synthetic risk signals.

Every algorithm is trained on *both* to quantify the uplift from source 2 and to interpret it alongside the leakage caveat.

5.2 Velocity features, encoding, and scaling

As required by the project, we add `txn_count_acct` (cumulative per-account transaction count, causal) and `time_since_last_txn` (gap to the previous transaction). The `type` variable is binary-encoded as `type_TRANSFER`; for Logistic Regression the numeric features are standardized with `StandardScaler`.

Practical observation. On PaySim, accounts almost never repeat: `txn_count_acct` has mean ≈ 1.0 and `time_since_last_txn` is mostly -1 . These features are built *as required* but carry very little signal (see Section 6.3); on a real platform with returning customers they would matter.

5.3 Handling class imbalance

We address imbalance with **class weights** in all three models (`class_weight="balanced"` for Logistic Regression; `scale_pos_weight $\approx$ 337` for XGBoost; `class_weight="balanced_subsample"` for Random Forest), combined with the type filtering that already raised positive density.

5.4 Feature selection

We rank features by *mutual information* (MI) on a subsample. A methodological point: *univariate* MI ranks `errorBalanceOrig` rather low, **contrary** to its dominant importance in tree models — because its strength lies in nonlinear interactions. We therefore **keep all features** for the tree models (no harm) rather than pruning by univariate MI, only noting the near-uninformative group (`newbalanceOrig`).

6 Model Development and Evaluation (Module 5)

6.1 Setup

We split train/test 70/30, stratified by label (train 1,939,275 samples / 5,738 fraud; test 831,118 / 2,459). We train **three algorithms** $\times$ **two feature spaces**: Logistic Regression, Random Forest, XGBoost. The primary metric is **PR-AUC** (suited to imbalanced data), with ROC-AUC, Recall, Precision, and F1 at the default threshold 0.5.

6.2 Comparison of results

Table 4 summarizes the 6 models.

Table 4: Comparison of 6 models on the test set (831,118 samples, 2,459 fraud). Threshold 0.5.

Model	PR-AUC	ROC-AUC	Recall	Precision	F1
LogReg · Base	0.5932	0.9782	0.9081	0.0433	0.0827
LogReg · Full	0.9704	0.9999	0.9947	0.3348	0.5010
RandomForest · Base	0.9974	0.9989	0.9967	0.9996	0.9982
RandomForest · Full	0.9996	1.0000	0.9963	1.0000	0.9982
XGBoost · Base	0.9944	0.9991	0.9923	0.9204	0.9550
XGBoost · Full	0.9997	1.0000	0.9976	0.9927	0.9951

Key observations:

- **Random Forest · Base is the best honest model:** PR-AUC 0.997 with only **1 false positive** out of 828,659 legitimate transactions (8 fraud missed) — using *no synthetic features at all*.

- **Synthetic features help the linear model greatly:** Logistic Regression Base reaches only PR-AUC 0.59 (precision 0.04, i.e., 48,850 false positives), but Full rises to 0.97.
- On the Base set, XGBoost already reaches 0.994 — nearly as good as Full — showing that PaySim is almost separable using balance features alone.

6.3 Feature importance

Table 5 lists XGBoost-Full importances. `errorBalanceOrig` dominates (0.537), matching the EDA; notably the synthetic flags have *low* importance and velocity is 0 — meaning the tree model relies mainly on *real* features, which tempers (but does not eliminate) the leakage concern.

Table 5: Feature importance (XGBoost Full, top 10 and tail group).

Feature	Importance	Feature	Importance
<code>errorBalanceOrig</code>	0.537	<code>amount</code>	0.013
<code>account_age_days</code>	0.157	<code>shipping_billing_mism.</code>	0.010
<code>newbalanceOrig</code>	0.121	<code>is_new_device</code>	0.007
<code>failed_payment_attempts</code>	0.079	<code>oldbalanceDest</code>	0.002
<code>ip_billing_mismatch</code>	0.028	<code>txn_count_acct</code>	0.000
<code>type_TRANSFER</code>	0.016	<code>time_since_last_txn</code>	0.000

6.4 Cost-based metric and operating-threshold selection

The default threshold 0.5 is not business-optimal. We turn the Precision/Recall trade-off into **money**: each missed fraud (FN) loses exactly the *transaction amount*; each false alarm (FP) costs a fixed friction $C_{FP} = 10$. The total cost at threshold t is

$$\text{Cost}(t) = \sum_{i: y_i=1, \hat{p}_i < t} \text{amount}_i + C_{FP} \cdot |\{i : y_i = 0, \hat{p}_i \geq t\}|. \quad (3)$$

Sweeping the threshold on XGBoost Base (the honest, deployment-suited model) gives an **optimal threshold** $t^* = 0.21$ (Table 6), reducing total cost by $\sim 16.6\%$ versus 0.5.

Table 6: Operating-threshold comparison (XGBoost Base, test set).

Threshold	Total cost	Recall	FP
0.50 (default)	1,628,601	0.992	211
0.21 (optimal)	1,358,351	0.997	260

Financial KPI at $t^* = 0.21$. The model stops $\approx 100\%$ of fraud money (3.6139B of 3.6152B on the test set), letting only $\sim 1.36\text{M}$ through, with an **FPR** = 0.031% (260 legitimate transactions wrongly blocked) — a well-balanced operating point between loss and friction.

6.5 Limitations and honest interpretation

The near-perfect scores (RF Base PR-AUC 0.997, 1 FP) should **not** be read as excellent capability, but as a sign that the data is *too easy*:

- PaySim simulates fraud with a *near-deterministic balance signature* (fraud drains the origin account), so all three tree models separate it using real features alone.
- The synthetic features are generated from the label, so the Full set carries **leakage**; Full’s scores should be seen as an *optimistic upper bound*, while the Base set is closer to real-world difficulty.

We therefore present *both* feature spaces and recommend **Random Forest Base** as the best honest model, and **XGBoost Base** as the deployment choice (compact, fast, no leakage).

7 Conclusion

We built an end-to-end fraud-detection pipeline on PaySim combined with synthetic data, covering all five modules: business understanding and KPIs, EDA, documented cleaning, feature engineering with imbalance handling and feature selection, and training/evaluation of six models with a cost-based metric. Technically, XGBoost/Random Forest reach PR-AUC > 0.99 and the operating threshold 0.21 stops nearly all fraud money at an FPR of 0.031%. Methodologically, the core contribution is the *Base-vs-Full comparison design* that exposes the label-leakage risk, and the conclusion that near-perfect scores reflect the easiness of the data rather than genuine generalization — a necessary honest perspective for reporting.

Future work (Modules 6–8): package the model as an API and review UI, monitor data drift, and translate findings into risk-policy recommendations.